AI BASED SHORT-TERM ELECTRICITY DEMAND FORECASTING USING OPTIMIZED RANDOM FOREST

Project Report submitted to Sri Ramakrishna College of Arts & Science for Women

in Partial fulfilment of the requirement for the award of the degree of

MASTER OF SCIENCE

DATA ANALYTICS

Submitted by

SREESANDHIYA S

252CF021

UNDER THE GUIDANCE OF

Dr.D.Anitha

ASSOCIATE PROFESSOR

SCHOOL OF COMPUTER SCIENCE

SRI RAMAKRISHNA COLLEGE OF ARTS & SCIENCE FOR WOMEN

(Autonomous)

Coimbatore – 641044.

OCTOBER 2026

SRI RAMAKRISHNA
COLLEGE OF ARTS & SCIENCE
FOR WOMEN

PROJECT WORK

AI BASED SHORT-TERM ELECTRICITY DEMAND FORECASTING USING OPTIMIZED RANDOM FOREST

Bonafide Work Done by

Sreesandhiya S (252CF021)

Project Report submitted to Sri Ramakrishna College of Arts & Science for Women in Partial fulfilment of the requirement for the award of the Degree

Master of Science

Data Analytics

Signature of Guide **Signature of HOD**

Submitted for Viva-Voce Examination held on: _____

Internal Examiner **External Examiner**

CERTIFICATE

This is to certify that the project entitled "**AI BASED SHORT-TERM ELECTRICITY DEMAND FORECASTING USING OPTIMIZED RANDOM FOREST"** submitted to the Sri Ramakrishna College of Arts & Science for Women, in partial fulfillment of the requirements for the award of **Master of Science** is a record of original project work done by Ms. Sreesandhiya S during the period of her study in **School of Computer Science (M.Sc Data Analytics)** at Sri Ramakrishna College of Arts & Science for Women, under my supervision and guidance and the dissertation has not formed the basis for the award of any Degree / Diploma / Associateship / Fellowship or other similar title to any candidate of any University.

Faculty Guide **Head of the Department**

DECLARATION

I SREESANDHIYA S hereby declare that this Capstone Project work titled **"AI BASED SHORT-TERM ELECTRICITY DEMAND FORECASTING USING OPTIMIZED RANDOM FOREST",** being submitted to the Sri Ramakrishna College of Arts & Science for Women, in partial fulfilment for the award of **Master of Science (Data Analytics)**, is the original project work carried out by me under the guidance of **Dr. D Anitha** and this project work has not formed the basis for the award of any Degree / Diploma / Associateship / Fellowship or other similar title to any candidate of any University.

(Signature of the Candidate)

Sreesandhiya S

252CF021

CONTENTS

CHAPTER NO.	CHAPTER SCHEME	PAGE NO
	ACKNOWLEDGEMENT	I
	SYNOPSIS	II
CHAPTER - 1	**INTRODUCTION**	01
	1.1 Project Overview	01
	1.2 Problem Statement	01
	1.3 Objectives	02
	1.4 Scope	02
CHAPTER - 2	**Literature survey**	03
	2.1 Related Work	03
	2.2 Existing Method	03
	2.3 Research Gap	04
CHAPTER - 3	**Methodology**	05
	3.1 Proposed Framework	05
	3.2 Data Collection	06
	3.3 Data Preprocessing	06
	3.4 Proposed Method	07
CHAPTER - 4	**Implementation**	08
	4.1 Dataset Description	08
	4.2 Module Description	08
CHAPTER – 5	**Results & Discussion**	10
	5.1 Metrics	10
CHAPTER – 6	**CONCLUSION**	12
	BIBLIOGRAPHY	13
	APPENDICES	11
	A. Sample Coding	14
	B. Screenshots	21

ACKNOWLEDGEMENT

First of all, I express my deep sense of gratitude to **Shri. Sundar Ramakrishnan, Managing Trustee, Sri Ramakrishna College of Arts & Science for Women**, for providing all the facilities in the College to carry out my project.

I express my profound sense of gratitude to **Dr. K. Chitra, M.Com., M.B.A., M.Phil., Ph.D., Principal, Sri Ramakrishna College of Arts & Science for Women**, for motivating me in all my endeavours.

I am grateful to **Dr. N. Tajunisha, MCA., M.Phil., Ph.D., Dean, School of Computer Science**, for extending her constant encouragement and valuable suggestions.

I express my sincere thanks to my Project Guide **Dr. D. Anitha, MCA., M.Phil., Ph.D., Associate Professor** for her valuable support, encouragement, and guidance rendered in completing my project.

My special thanks to all the Faculty Members of the **School of Computer Science**, for their strong support and suggestions in completing the project.

My heartfelt thanks to my family members, my beloved friends, and well-wishers, who have contributed directly and indirectly to the completion of the project work.

Above all, I thank the Almighty; with those abundant graces, this project work was made possible.

SYNOPSIS

A data-driven web application developed to analyze and forecast household electricity demand using historical electricity consumption data and machine learning techniques. It uses a Random Forest–based forecasting approach and incorporates electrical and temporal features such as reactive power, voltage, global intensity, sub-metering values, hour, day, month, and year.The system also includes a personalized meter-reading monitoring component. User-entered meter readings are stored and compared with previous consumption patterns. When the current daily usage is significantly higher than the user's historical average, the system generates High or Moderate Usage Alerts and sends email notifications to the registered email address.

The application is developed using Python and Django. SQLite was used during the initial development phase, and MySQL database configuration was also prepared as part of the database development process. The application currently includes user authentication, demand prediction, prediction history, meter-reading storage, notifications, and email alert functionality.The implemented workflow combines machine-learning-based electricity demand forecasting with user-level consumption monitoring and early warning notifications, helping users track and manage their electricity usage more effectively.

CHAPTER 1

INTRODUCTION

1.1 Project Overview

Electricity demand forecasting is an important data analytics problem because household electricity consumption changes according to electrical conditions and time-related patterns. Smart Energy Analytics is designed as a web-based analytics system that uses historical electricity consumption data to estimate electricity demand and also monitors the user's actual meter readings.

The forecasting component accepts ten features: Global Reactive Power, Voltage, Global Intensity, Sub Metering 1, Sub Metering 2, Sub Metering 3, Hour, Day, Month and Year. A trained Random Forest model processes these features and produces a predicted electricity demand value.

In addition to forecasting, the system provides personalized meter-reading monitoring. The current meter reading is compared with previous readings for the same user. Consumption is normalized by the number of days between readings so that the system can compare daily usage rather than only comparing absolute meter values. Based on the user's historical daily average, the system generates Moderate or High Usage Alerts and sends an email to the registered email address.

1.2 Problem Statement

Conventional electricity applications commonly focus on displaying or calculating electricity consumption or bill values. Such approaches do not necessarily provide personalized early awareness when a particular user's consumption starts increasing compared with their own normal pattern. A data analytics based system is therefore required to combine electricity demand forecasting with user-specific consumption monitoring and alerts.

1.3 Objectives

- To develop a web-based household electricity demand forecasting system using electrical and temporal features with a Random Forest based machine learning model.
- To maintain individual user prediction history and store user-entered electricity meter readings with corresponding dates.
- To compare current daily electricity consumption with the user's historical average and generate Moderate or High Usage Alerts when significant increases are detected.
- To send personalized electricity usage alerts to the registered user's email address for early awareness and corrective action.

1.4 Scope

The present implementation focuses on household electricity demand forecasting and personalized electricity usage monitoring. The current scope includes user authentication, prediction, prediction history, meter reading, notification generation and email alerts. Future enhancements may include OCR-based bill upload, smart meter integration, mobile application support, AI chatbot support, carbon footprint analysis and weather-based prediction. These enhancements are considered future scope and are not treated as completed modules in the current implementation.

CHAPTER 2

LITERATURE SURVEY

2.1 Related Work

Recent research on electricity demand forecasting has explored classical statistical models, machine learning, ensemble methods, deep learning, probabilistic forecasting and practical web deployment. The studies below are reviewed based on their datasets, modelling approaches, evaluation methods and practical contributions.

[1] A Moving-Horizon Dynamic Mode Decomposition approach was proposed for daily residential electricity consumption prediction. The method continuously updates the historical data used for model training after each prediction period, highlighting the importance of adapting to changing residential demand patterns.

[2] A machine-learning-powered energy consumption prediction system was developed with a web application and API. The study compared Random Forest, Extra Trees, Support Vector Regression and XGBoost using timestamp, temperature, humidity and weather-related features. Extra Trees achieved an R^2 of 0.7441, MAPE of 16.27% and RMSE of 11.75 in the reported experiment. The work is relevant because it demonstrates practical deployment of an energy prediction model through a web interface.

[3] The selected base paper proposes an Optimized Random Forest framework for short-term electricity demand forecasting. It uses temperature and previous-day electricity demand as inputs and applies Grid Search to tune Random Forest hyperparameters. The study compares Random Forest, optimized Random Forest, Seasonal ARIMA and Exponential Smoothing, and reports MSE = 0.01 and R^2 = 0.894 for the optimized Random Forest in the reported winter evaluation.

2.2 Existing Method

Existing electricity demand forecasting methods can be broadly grouped into statistical, machine learning, ensemble and deep-learning approaches.

Classical methods such as Exponential Smoothing, ARIMA, ARMA and Kalman Filtering are relatively simple and computationally lighter. However, the base paper [3] notes that classical approaches may not capture complex nonlinear electricity-demand relationships as effectively as optimized machine-learning methods.

Machine-learning methods such as Random Forest, Support Vector Machine and regression models can learn nonlinear relationships from multiple input variables. Random Forest is useful because it combines several decision trees and provides robust regression predictions.

Deep-learning methods such as LSTM and CNN-BiLSTM can model temporal dependencies and are useful when large time-series datasets are available. Probabilistic approaches such as the Quantile LSTM framework in [5] additionally estimate uncertainty instead of only producing a single point prediction.

The base paper [3] specifically uses Random Forest with Grid Search hyperparameter optimization and compares it with standard RF, SARIMA and Exponential Smoothing. The present implementation follows this optimized Random Forest concept, but uses a different household electricity dataset, a different feature set and an 80/20 train-test split. The present system also differs at the application level. In addition to forecasting electricity demand, it stores user prediction history, records actual meter readings, compares current daily usage against historical daily usage and generates Moderate/High Usage Alerts with email notification

2.3 Research Gap

Although existing studies have achieved strong forecasting accuracy, the studies reviewed mainly concentrate on the forecasting model, dataset, or anomaly-detection method. The reviewed literature indicates several practical gaps.

[1], [3] and [5] primarily focus on forecasting methodology, while [4] focuses on anomaly detection. These approaches do not provide the complete user-facing workflow implemented in the present project, where prediction history and actual meter readings are maintained for an individual user.

Another gap is the difference between model performance and practical deployment. The work in [2] demonstrates web/API deployment, while the present project combines prediction with personalized meter monitoring and usage alerts in the same Django application.

Reported performance values cannot be compared directly across studies when the datasets, target units, forecasting horizons and evaluation protocols differ. Therefore, the present project compares its R^2 value with the base paper's R^2 only as a common metric and treats MAE/RMSE as project-specific evaluation results.

The proposed project addresses these gaps by implementing an optimized Random Forest forecasting model with GridSearchCV and integrating it with prediction history, meter-reading monitoring, historical usage comparison, Moderate/High alerts and automated email notifications.

CHAPTER – 3

METHODOLOGY

3.1 Proposed Framework

The proposed system is a web-based household electricity demand forecasting and monitoring application developed using Python and Django. It combines machine-learning-based electricity demand forecasting with personalized meter-reading monitoring.

The framework consists of two main components. The first component uses an Optimized Random Forest Regression model to forecast electricity demand based on electrical and temporal features such as Global Reactive Power, Voltage, Global Intensity, Sub Metering 1, Sub Metering 2, Sub Metering 3, Hour, Day, Month and Year.

The second component monitors the user's actual electricity consumption. Users can enter their meter reading and reading date. The system stores these readings and compares the current daily usage with the user's historical average. Based on the comparison, the system generates Moderate Usage or High Usage alerts when consumption exceeds the defined threshold. The alert is stored in the database and an email notification is sent to the registered user's email address.

The overall workflow is:

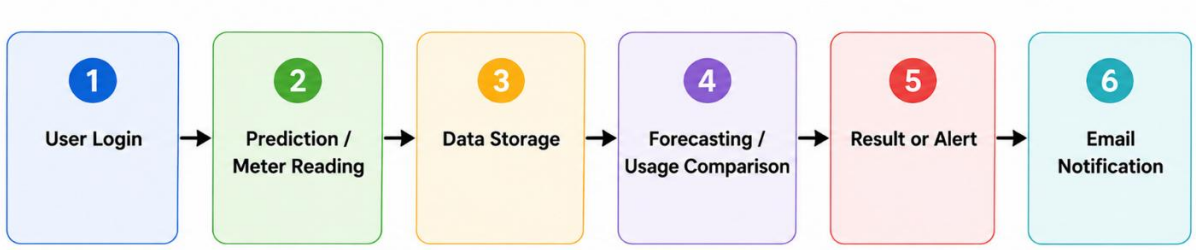


Fig.3 Proposed Framework

Thus, the proposed framework provides both electricity demand forecasting and personalized consumption monitoring in a single web application.

3.2 Data Collection

The dataset used for electricity demand forecasting was obtained from the Kaggle Household Electric Power Consumption Dataset, which originates from the UCI Individual Household Electric Power Consumption Dataset.

The original dataset contains 2,075,259 rows and 9 columns. It includes historical household electricity measurements such as Date, Time, Global Active Power, Global Reactive Power, Voltage, Global Intensity, Sub Metering 1, Sub Metering 2, and Sub Metering 3. The dataset contains missing or incomplete values, which are removed during preprocessing. After removing the missing records, 2,049,280 complete records are used for analysis and model development.

The Date and Time fields are used to derive the temporal features Hour, Day, Month, and Year. Global Active Power is used as the target variable for electricity demand forecasting. Thus, the final model dataset consists of 10 input features and 1 target variable (11 columns).

The processed dataset is divided into 80% training data and 20% testing data. The training data is used for model development and optimization, while the testing data is used to evaluate the forecasting performance.

Dataset Source:
Kaggle – Household Electric Power Consumption Dataset.

User-entered meter readings are collected separately through the Django application for personalized electricity consumption monitoring.

3.3 Data Preprocessing

Preprocessing is performed to convert the collected electricity data into a suitable format for machine learning.

The main preprocessing steps are:

- **Data Loading:** The dataset is loaded using Python and Pandas.
- **Date and Time Processing:** The Date and Time fields are processed to obtain useful temporal information.
- **Feature Extraction:** Hour, Day, Month and Year are extracted from the date and time information.

- **Feature Selection:** The required electrical and temporal features are selected as inputs for the forecasting model.
- **Data Preparation:** The selected features are organized as input variables, while the electricity demand value is used as the target.
- **Train-Test Split:** The prepared data is divided into training and testing sets using an 80:20 ratio.

The processed data is then supplied to the Random Forest model for training and optimization.

3.4 Proposed Method

The proposed method uses an Optimized Random Forest Regression algorithm for short-term electricity demand forecasting.

Random Forest is an ensemble machine learning technique that combines multiple decision trees to produce a final prediction. It is suitable for this project because electricity demand can have complex and nonlinear relationships with electrical and temporal features.

In the proposed system, GridSearchCV is used to optimize the Random Forest hyperparameters. Different parameter combinations are evaluated and the best-performing configuration is selected.

Parameter	**Value**
n_estimators	200
max_depth	20
min_samples_split	5
min_samples_leaf	2

Table 3.1 Optimized Random Forest Hyperparameters

After optimization, the model is trained using the preprocessed dataset and integrated into the Django application for demand prediction.The method follows the selected base paper’s optimized Random Forest approach with the project’s own dataset, features, and application workflow.

CHAPTER 4

IMPLEMENTATION

4.1 Dataset Description

The household electricity consumption dataset used in this project was obtained from Kaggle and originates from the UCI Individual Household Electric Power Consumption repository. It contains electrical measurements recorded over time and was used as the primary source for training and evaluating the electricity demand forecasting model.

The forecasting interface uses ten input features: Global Reactive Power, Voltage, Global Intensity, Sub Metering 1, Sub Metering 2, Sub Metering 3, Hour, Day, Month and Year. These electrical and temporal features are provided to the optimized Random Forest model for predicting electricity demand.

Dataset links:Kaggle | UCI

Dataset Source: Kaggle – Household Electricity Consumption Dataset.

4.2 Module Description

Login and Registration: Provides secure user registration and login using Django authentication. User details and registered email are stored using Django's built-in User model. The email address is used for sending personalized electricity usage alerts. After login, users can access their personal system features.

Electricity Demand Prediction : Accepts the ten input features required by the Random Forest forecasting model.The entered values are processed and passed to the optimized Random Forest model.The model predicts the expected electricity demand based on the input values. The prediction result is displayed through the Django web application.

Prediction History: Stores the electricity demand predictions generated by each user. Prediction details are associated with the currently logged-in user. Users can view their previous prediction results and dates. This helps users monitor their forecasting history over time.

Meter Reading: Allows users to enter their current electricity meter reading and reading date. The entered readings are stored securely and linked to the respective user. The latest meter reading is displayed after successful submission. Multiple readings are used for personalized electricity consumption analysis.

Personalized Usage Comparison: Calculates electricity units consumed between two consecutive meter readings.The consumption is normalized based on the number of days between readings.Current daily usage is compared with the user's historical daily usage. This helps identify increases or decreases in personal electricity consumption.

Notifications: Generates personalized alerts based on the user's electricity consumption level. The system provides High and Moderate usage notifications. Generated alerts are stored in the database for the logged-in user. Users can view these alerts through the Django web application.

Email Alert: Sends electricity usage alerts to the registered email address of the user. Alerts are generated when consumption reaches High or Moderate levels. The email provides timely information about the detected usage condition. This helps users monitor electricity consumption without opening the application.

Database Implementation: Django models are used to store and manage application data. PredictionHistory stores prediction results, while MeterReading stores meter data. Notification stores usage alerts, and Django User manages user accounts. Database migrations create and maintain the required database structure.

CHAPTER 5

RESULTS & DISCUSSION

5.1 Metrics

The optimized Random Forest model was evaluated on the testing data. The current recorded evaluation values are:

Metric	Present Project
R^2 Score	0.999420
MAE	0.012713
RMSE	0.027257

Table 5.1 Present project evaluation metrics

The model development also included GridSearchCV based hyperparameter optimization. The selected parameter combination was n_estimators = 200, max_depth = 20, min_samples_split = 5 and min_samples_leaf = 2.

The web application was tested for the personalized meter-reading workflow. A clean test sequence of readings was used: 1000 kWh on 13 September 2026, 1100 kWh on 14 September 2026 and 1300 kWh on 15 September 2026. The resulting daily usages were 100 kWh/day and 200 kWh/day. The current usage was therefore detected as significantly higher than the historical average, a High Usage Alert was created and an email was successfully delivered to the registered email account.

The result demonstrates the distinction between the forecasting and monitoring components. Forecasting estimates electricity demand from electrical and temporal features, whereas meter monitoring compares an individual user's actual consumption with their own historical pattern and provides an early warning.

The error metrics also require careful interpretation. The base paper reports MSE, whereas the present project reports MAE and RMSE; therefore these values are not directly comparable without using the same metric, dataset and experimental protocol. The comparison is included to show how the implemented model performs relative to the study that motivated the proposed approach.

The present project uses a household electricity consumption dataset, ten electrical and temporal input features, and an 80/20 train-test split. The implemented model obtained $R^2 = 0.999420$, MAE = 0.012713 and RMSE = 0.027257. The common metric, R^2, is higher in the present project by 0.105420 (approximately 11.79% relative to the base-paper R^2). However, this difference should not be interpreted as proof that the present model is universally better, because the datasets, target variables, feature sets, forecasting horizons, units and evaluation protocols are different.

The present implementation is based on the optimized Random Forest concept reported in [3], but the experimental setup is different. The base paper evaluates optimized Random Forest using electricity demand and temperature data from an Iranian distribution substation, uses an 85% training and 15% testing split, and reports MSE and R^2 across representative seasonal days. Its optimized Random Forest achieved MSE = 0.01 and $R^2 = 0.894$ in the reported winter evaluation.

Metric	Base Paper [3]	Present Project	Difference / Interpretation
R^2	0.894	0.999420	+0.105420; present project is higher
MSE	0.01	Not reported	Not directly compared because present project reports RMSE/MAE
MAE	Not reported	0.012713	Project-specific metric
RMSE	Not reported	0.027257	Project-specific metric

Table 5.2 Comparison of reported evaluation metrics

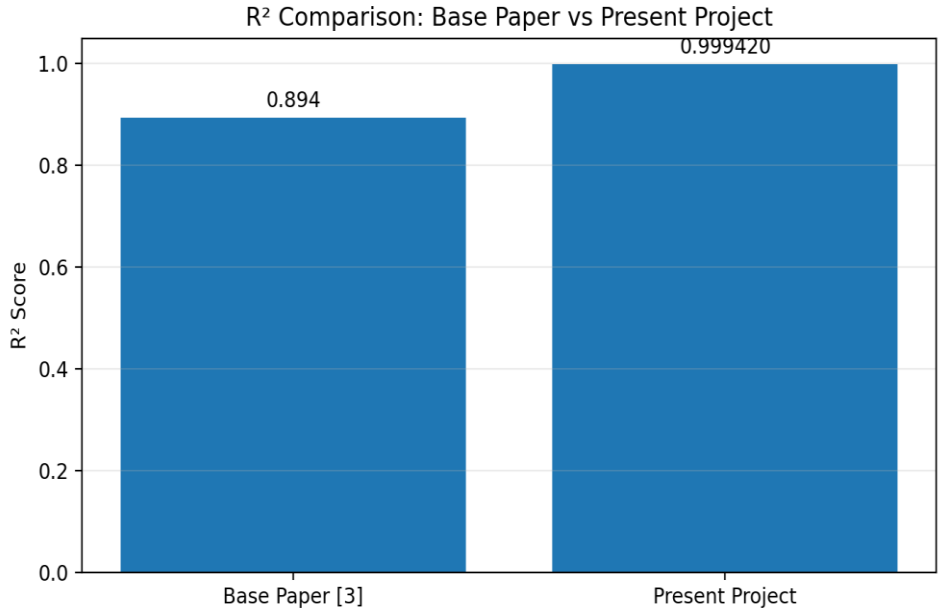


Figure 5.1 R^2 comparison between the base paper and the present project

CHAPTER 6

CONCLUSION

AI Based Short-Term Electricity Demand Forecasting Using Optimized Random Forest was developed as a Django-based household electricity analytics application combining machine learning based electricity demand forecasting with personalized meter-reading monitoring. The Random Forest forecasting model was trained using electrical and temporal features and optimized using GridSearchCV. The current implementation achieved the recorded evaluation values of R^2 0.999420, MAE 0.012713 and RMSE 0.027257 on the project test evaluation.

The personalized meter-reading component extends the system beyond prediction. It stores user readings, calculates daily consumption, compares current usage with the user's historical daily average and generates Moderate or High Usage Alerts. Email alert functionality was successfully integrated using Django email configuration and tested with the registered user email.

The current implementation provides the foundation for further development of personalized energy analytics. Future improvements can include more diverse datasets, OCR-based bill extraction, smart-meter integration, chatbot support and weather-aware forecasting.

BIBLIOGRAPHY

[1] D. H. Nguyen, “Daily Residential Electric Consumption Prediction Using Moving-Horizon Dynamic Mode Decomposition,” IEEE Access, vol. 12, 2024.

[2] T. A. Abd’Azeez and L. Olatomiwa, “Machine Learning-Based Energy Consumption Prediction System,” J. Electr. Syst. Inf. Technol., vol. 12, 2025.

[3] H. Lotfi et al., “Optimized Machine Learning for Short-Term Electricity Demand Forecasting,” Discover Applied Sciences, vol. 7, 2025.

[4] A. C. Vishnu Dharssini et al., “Anomergy: Data-Driven Framework for Residential Energy Anomaly Detection,” e-Prime, vol. 13, 2025.

[5] Z. Masood et al., “Short-Term Household Load Forecasting Using Quantile LSTM,” IEEE Access, vol. 12, 2024.

[6] S. R. Gaikwad et al., “Household Electricity Consumption Prediction Using Ensemble and Hybrid Models,” Scientific Reports, vol. 14, 2024.

[7] M.-P. Wu and F. Wu, “Residential Electricity Consumption Prediction Using CNN-BiLSTM-SA,” IEEE Access, vol. 12, 2024.

[8] E. Demir and S. Gunal, “Short-Term Electricity Consumption Forecasting with Deep Learning,” J. Supercomputing, vol. 81, 2025.

[9] L. Wang et al., “Short-Term Residential Electricity Consumption Forecast Considering Temperature Effects,” Energy Informatics, vol. 8, 2025.

[10] M. Neshat et al., “Smart Building Energy Consumption Forecasting Using Adaptive Evolutionary Models,” Energy, vol. 333, 2025.

APPENDICES

A. Sample Coding

predictor/views.py

```
from django.shortcuts import render
from django.contrib.auth.decorators import login_required
from django.db.models import Avg, Max
from django.conf import settings
from django.core.mail import send_mail
from datetime import datetime
from .ml_model import predict_demand
from .models import PredictionHistory, MeterReading, Notification
@login_required(login_url='login')
def home(request):
    prediction = None
    if request.method == "POST":
        data = [
float(request.POST["global_reactive_power"]),
float(request.POST["voltage"]),
float(request.POST["global_intensity"]),
float(request.POST["sub_metering_1"]),
float(request.POST["sub_metering_2"]),
float(request.POST["sub_metering_3"]),
int(request.POST["hour"]),
            int(request.POST["day"]),
            int(request.POST["month"]),
            int(request.POST["year"])]
        prediction = predict_demand(data)
        PredictionHistory.objects.create(
```

```
            user=request.user,
            global_reactive_power=data[0],
            voltage=data[1],
            global_intensity=data[2],
            sub_metering_1=data[3],
            sub_metering_2=data[4],
            sub_metering_3=data[5],
            hour=data[6],
            day=data[7],
            month=data[8],
            year=data[9],
            prediction=prediction
        ) return render(request, "index.html", {
        "prediction": prediction     })
@login_required(login_url='login')
def meter_reading(request):
    if request.method == "POST":
        reading = float(request.POST["reading"])
        reading_date = datetime.strptime(
            request.POST["reading_date"],
            "%Y-%m-%d"
        ).date()
        # Save current reading
        current_reading = MeterReading.objects.create(
            user=request.user,
            reading=reading,
            reading_date=reading_date )
        # Get previous reading
        previous_reading = MeterReading.objects.filter(
```

```
user=request.user
).exclude(
    id=current_reading.id
).order_by(
    "-reading_date",
    "-created_at"
).first()
# Continue only if previous reading exists
if previous_reading:
    # Calculate consumed units
    units_consumed = (
        reading - previous_reading.readin  )
    # Calculate number of days
    days = (
        current_reading.reading_date -
        previous_reading.reading_date
    ).days
    if days <= 0:
        days = 1
    # Daily electricity usage
    daily_usage = units_consumed / days
    # Get older readings
    older_readings = MeterReading.objects.filter(
        user=request.user
    ).exclude(
        id=current_reading.id
    ).order_by(
        "-reading_date",
        "-created_at"   )
```

```
# Calculate historical daily usage
historical_daily_usage = []
readings_list = list(older_readings)
for i in range(len(readings_list) - 1):
    newer = readings_list[i]
    older = readings_list[i + 1]
    old_units = (
        newer.reading -
        older.reading
    )
    old_days = (
        newer.reading_date -
        older.reading_date
    ).days
    if old_days <= 0:
        old_days = 1
    old_daily_usage = (
        old_units / old_days   )
    if old_daily_usage >= 0:
        historical_daily_usage.append(
            old_daily_usage )
# Check if historical usage is available
if historical_daily_usage:
    average_daily_usage = (
        sum(historical_daily_usage) /
        len(historical_daily_usage)   )
    # HIGH USAGE
    if daily_usage > average_daily_usage * 1.20:
        message = (
```

```
            f"Your daily electricity usage is "
            f"{daily_usage:.2f} kWh/day, which is "
            f"significantly higher than your usual "
            f"average of {average_daily_usage:.2f} "
            f"kWh/day." )
        Notification.objects.create(
            user=request.user,
            message=message,
            status="High" )
        # Send email
        if request.user.email:
            send_mail(
                "High Electricity Usage Alert",
                message,
                settings.DEFAULT_FROM_EMAIL,
                [request.user.email],
                fail_silently=False, )
    # MODERATE USAGE
    elif daily_usage > average_daily_usage * 1.10:
        message = (
            f"Your daily electricity usage is "
            f"{daily_usage:.2f} kWh/day, which is "
            f"slightly higher than your usual "
            f"average of {average_daily_usage:.2f} "
            f"kWh/day." )
        Notification.objects.create(
            user=request.user,
            message=message,
            status="Moderate" )
```

```
                # Send email
                if request.user.email:
                    send_mail(
                        "ι Moderate Electricity Usage Alert",
                        message,
                        settings.DEFAULT_FROM_EMAIL,
                        [request.user.email],
                        fail_silently=False, )
    # Show latest reading
    latest_reading = MeterReading.objects.filter(
        user=request.user
    ).order_by(
        "-reading_date",
        "-created_at"
    ).first()
    return render(
        request,
        "meter_reading.html",{
            "latest_reading": latest_reading  } )
@login_required(login_url='login')
def dashboard(request):
    records = PredictionHistory.objects.filter(
        user=request.user
    ).order_by('-created_at')
    total_predictions = records.count()
    average_prediction = records.aggregate(
        Avg('prediction')
    )['prediction__avg']
    highest_prediction = records.aggregate(
```

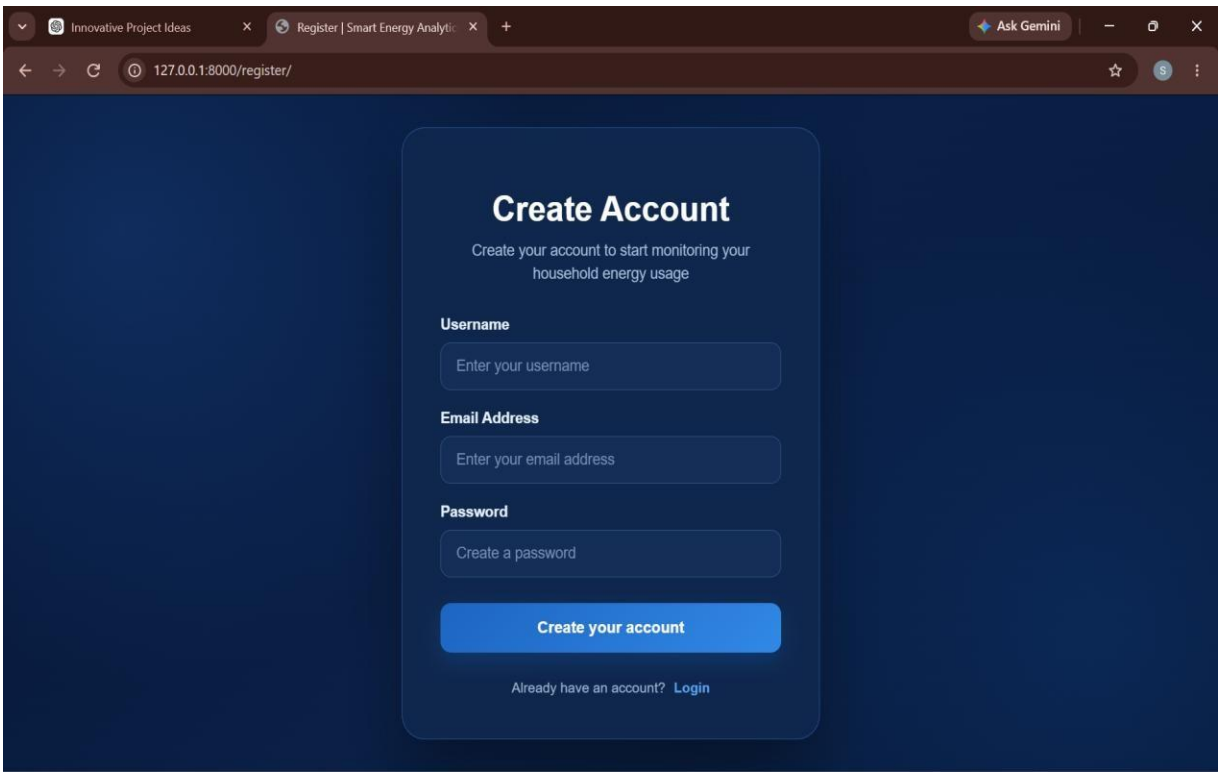
```
    Max('prediction')
    )['prediction__max']
    latest_prediction = records.first()
    # Last 7 predictions
    recent_predictions = list(records[:7])

    recent_predictions.reverse()
    chart_labels = [
        item.created_at.strftime('%d-%m')
        for item in recent_predictions  ]
    chart_data = [
        float(item.prediction)
        for item in recent_predictions  ]
    context = {
        'total_predictions': total_predictions,
        'average_prediction': average_prediction,
        'highest_prediction': highest_prediction,
        'latest_prediction': latest_prediction,
        'recent_predictions': recent_predictions,
        'chart_labels': chart_labels,
        'chart_data': chart_data,
        'bar_labels': chart_labels,
        'bar_data': chart_data,   }
    return render(
        request,
        'dashboard.html',
        context  )
@login_required(login_url='login')
def history(request):
```

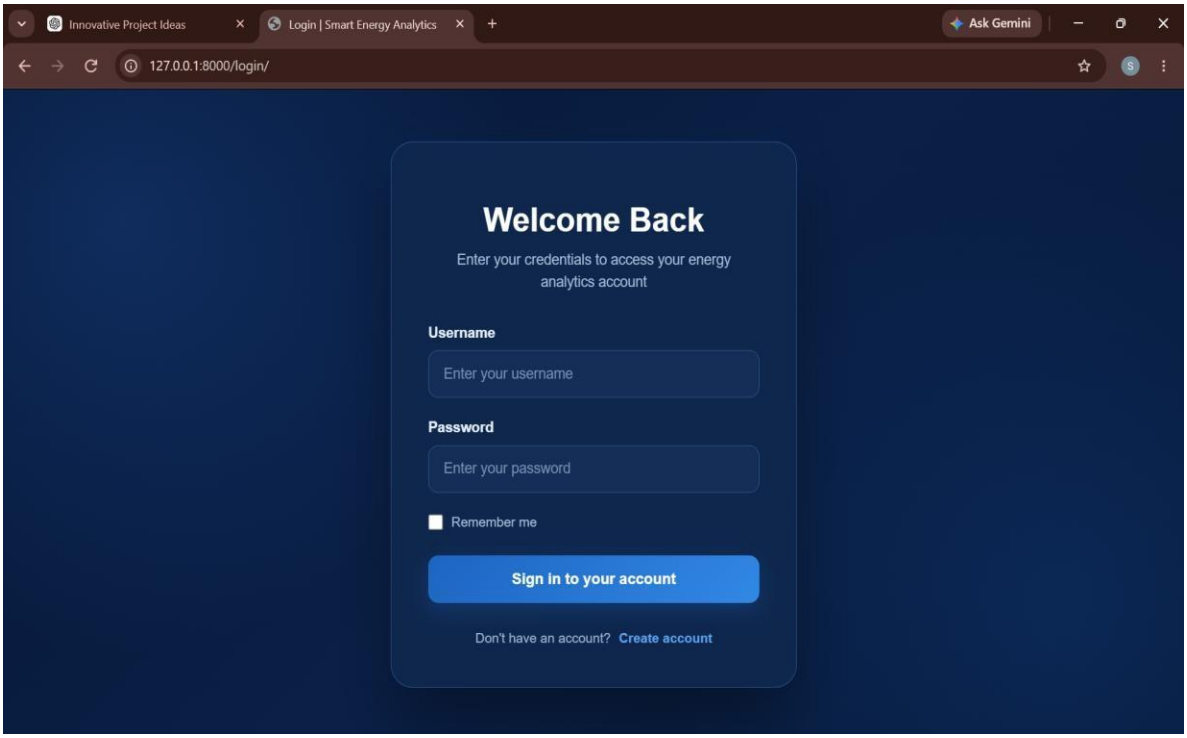
```
    predictions = PredictionHistory.objects.filter(
        user=request.user
    ).order_by("-created_at")
    return render(
        request,
        "history.html", {
            "predictions": predictions  } )
@login_required(login_url='login')
def notifications(request):
    alerts = Notification.objects.filter(
        user=request.user
    ).order_by("-created_at")
    return render(
        request,
        "notifications.html",  {
            "alerts": alerts  })
```

B. Screenshots

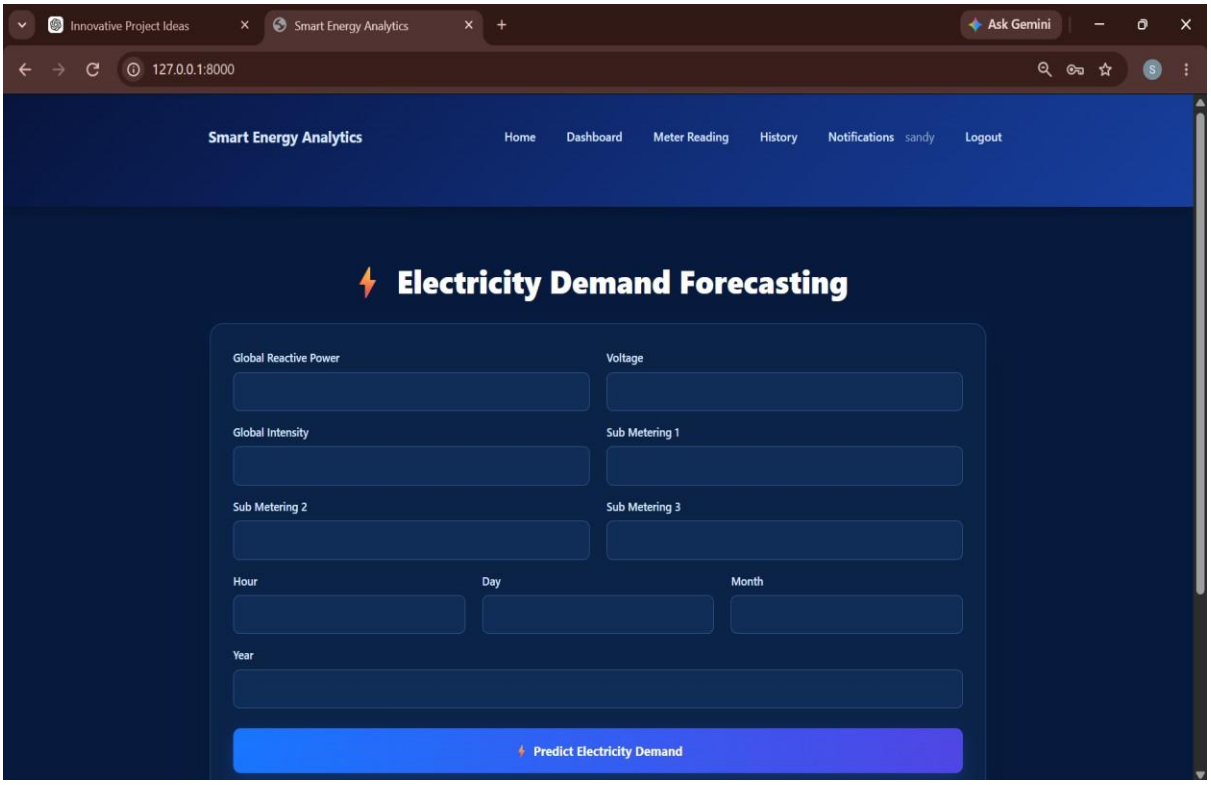
• Login page



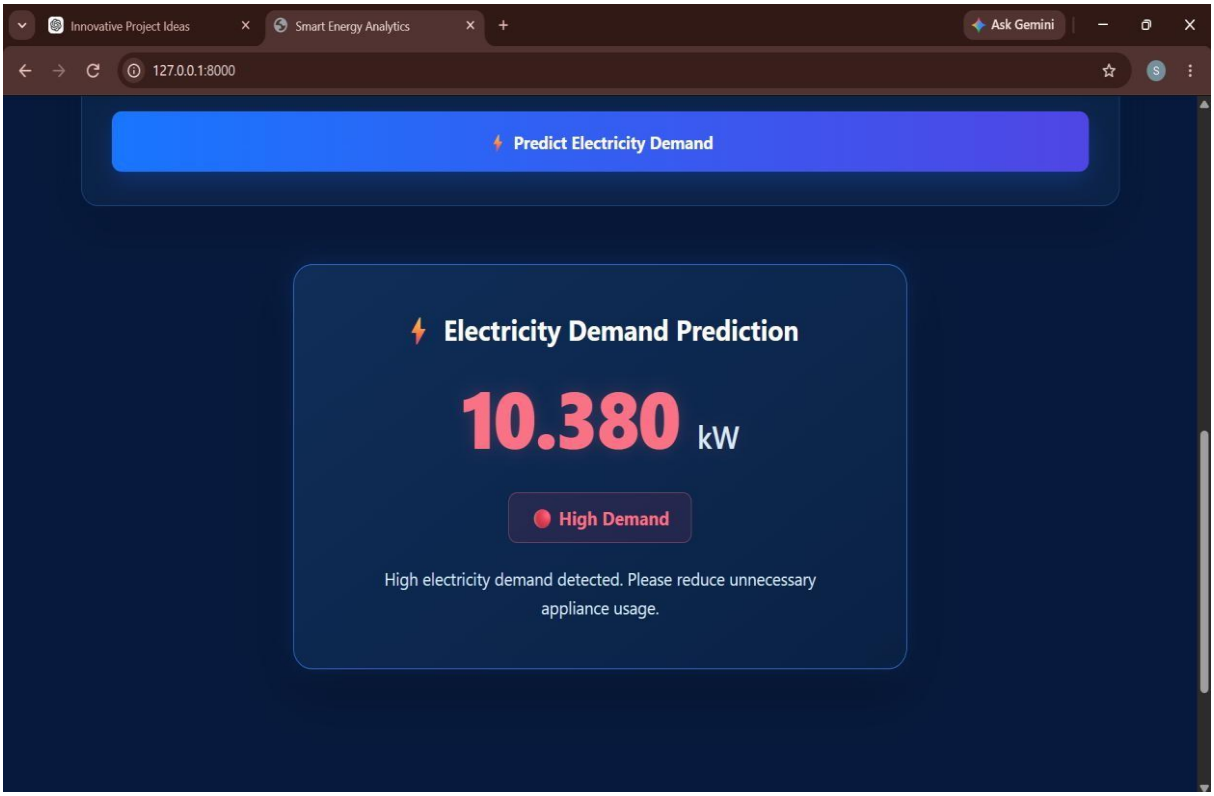
• Registration page



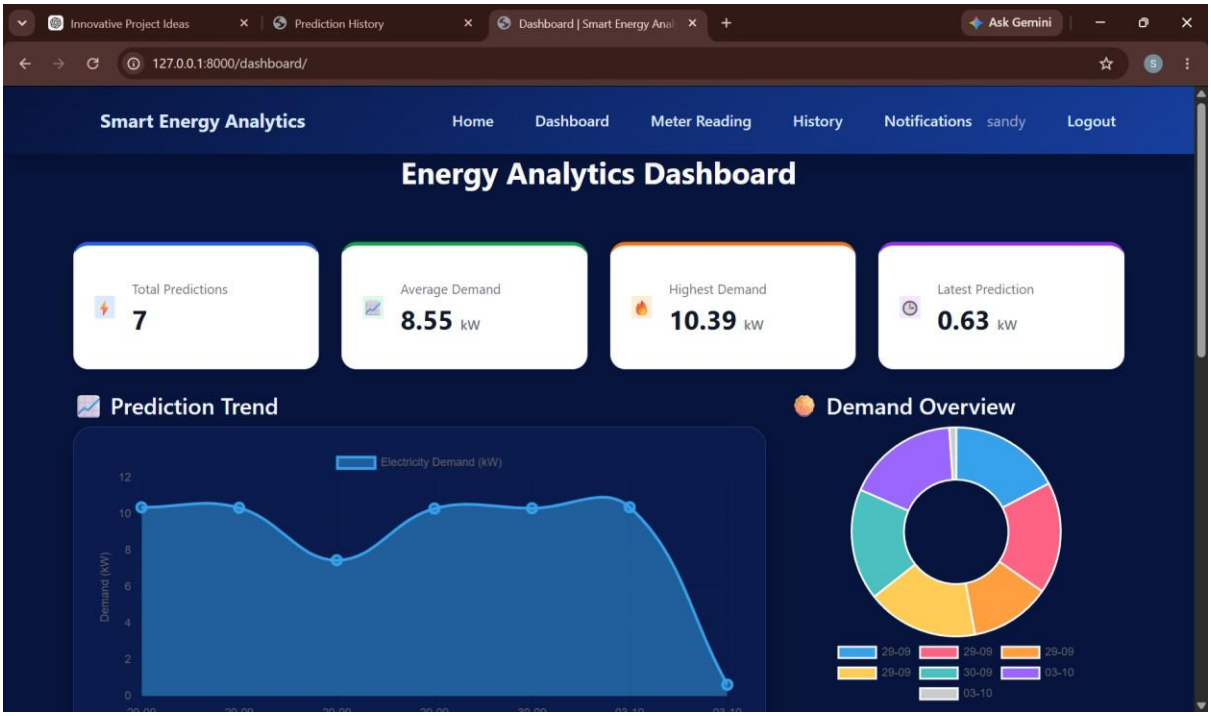
• Electricity Demand Forecasting input page

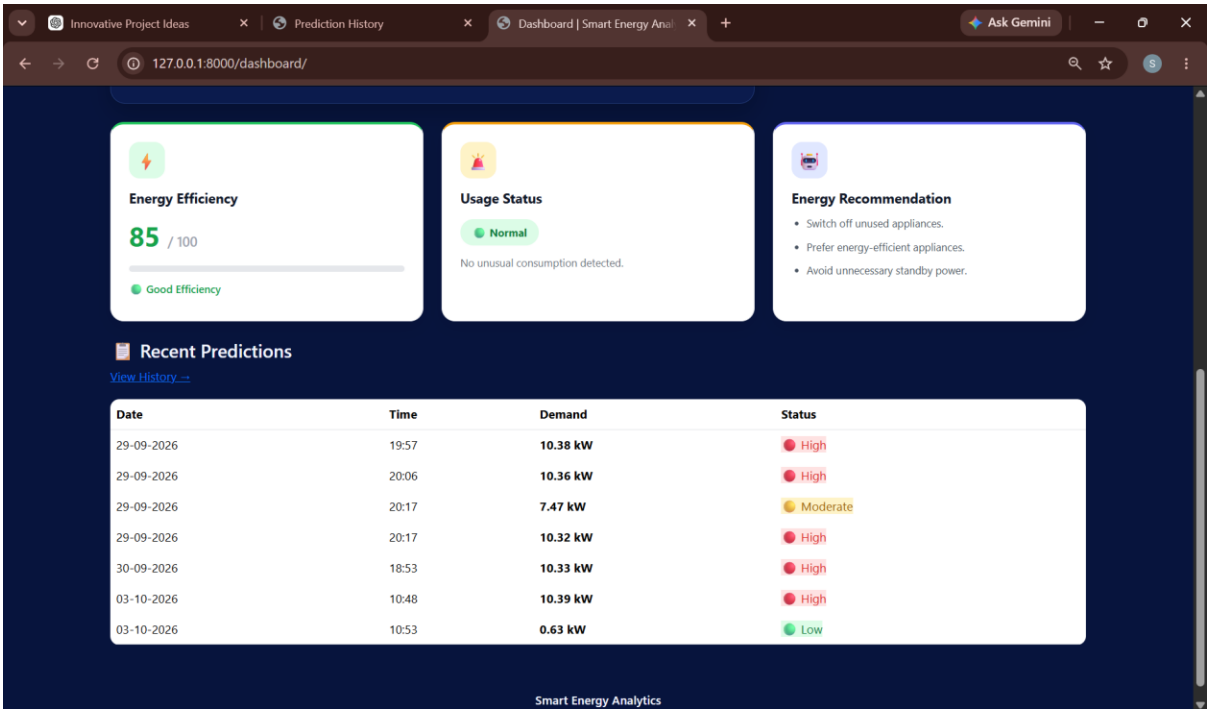


• Prediction result page

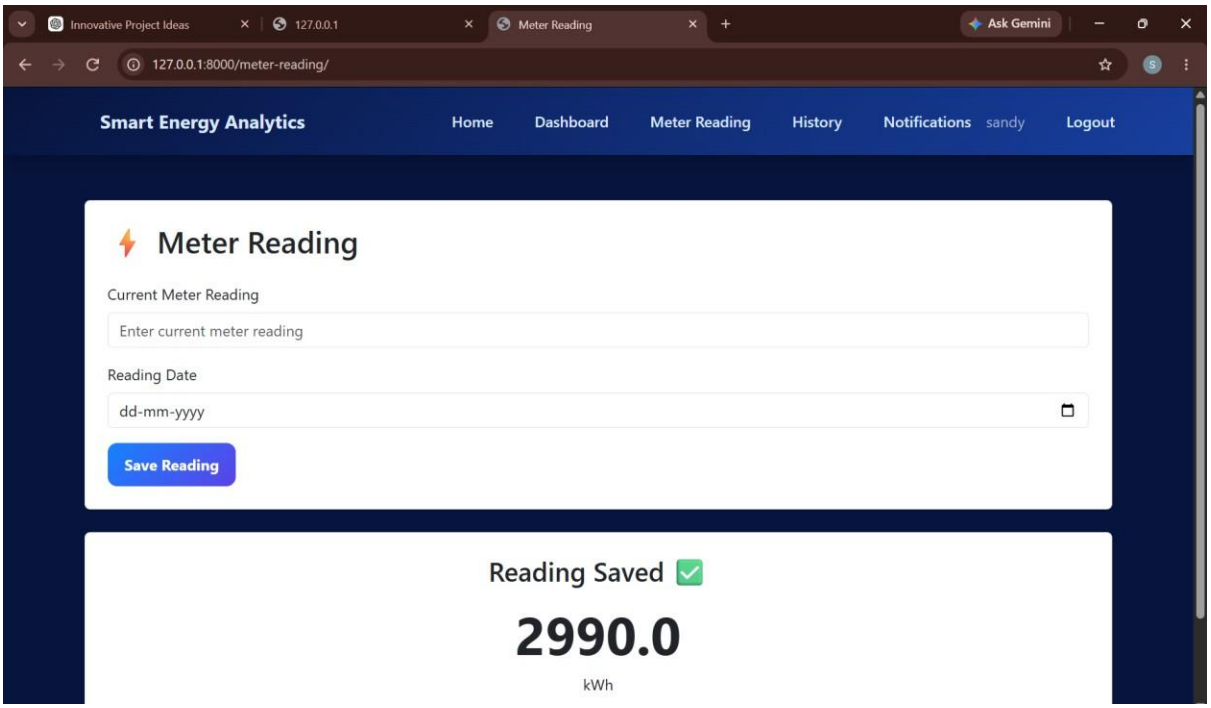


• Project Dashboard

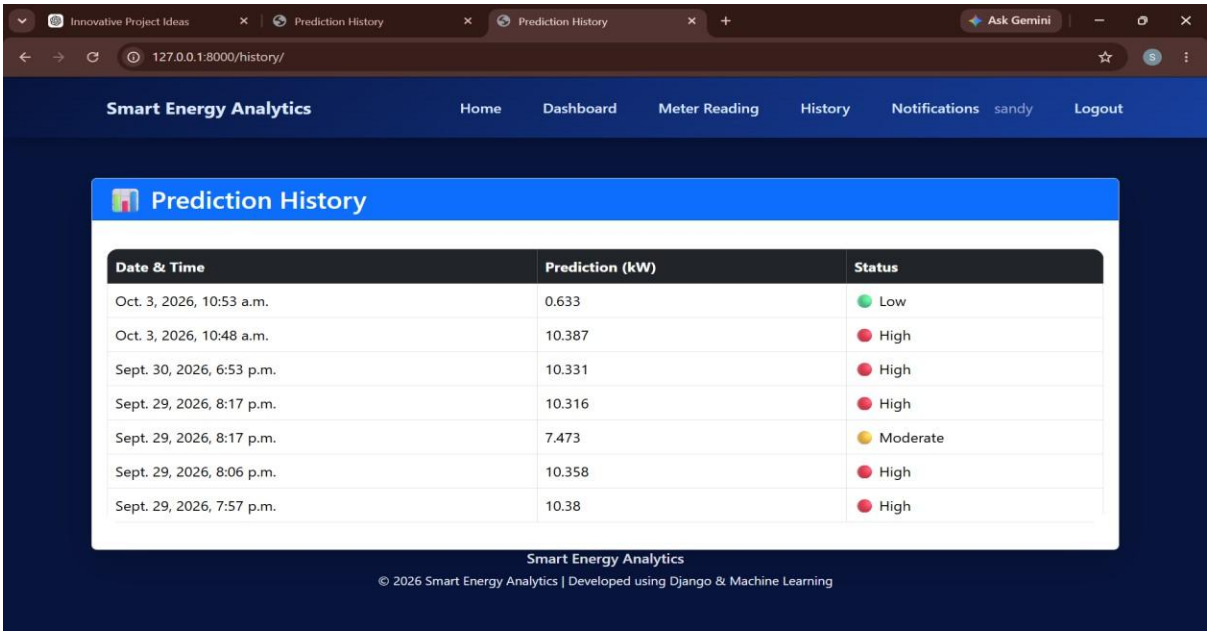




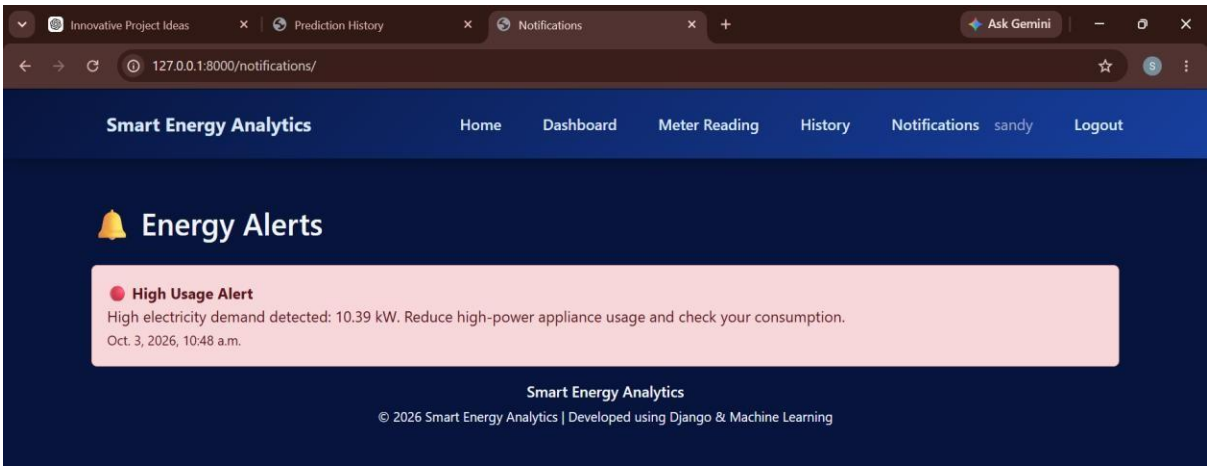
• Meter Reading page



• Prediction history page



• Notification page



• High Usage Alert email received in Gmail

